

7번 소인수분해 RSA 부분 비트 유출 복원

암호분석경진대회 제출 보고서

2026년 7월 28일

1 결과 요약

복원한 big-endian 평문 바이트열은 다음 ASCII 문자열이다.

```
FLAG{d1rty_b1t_l34k_c0pp3rsm1th_m33ts_str4t3gy}
```

동일한 바이트열의 hexadecimal 표현은 다음과 같다.

```
464c41477b64317274795f6231745f6c33346b5f633070703372736d3174685f  
6d333374735f73747234743367797d
```

공격은 양끝의 미지 4비트씩을 합친 256개 edge 후보를 계획하고, 나머지 미지 영역을 155비트와 230비트의 두 연속 변수로 완화한다. 각 후보에 대해 171차원 Herrmann-May 격자를 만들고 FLATTER로 감축한 뒤, 짧은 행에서 얻은 보조 다항식들의 modular resultant와 GCD를 계산하고 CRT로 근을 복원한다. 실제 재검증한 cid=155에서 두 소인수를 얻었고,

$$pq = N, \quad (p \& \text{MASK}) = \text{P_AND_MASK}, \quad m^e \bmod N = ct$$

를 모두 확인했다.

저장소의 runner는 cid=0,...,255를 숫자 순서로 모두 실행하도록 작성되어 있다. 다만 이 보고서는 전체 256개 blind scan을 완료했다고 주장하지 않는다. pinned 환경에서 실제로 다시 확인한 공격 branch는 cid=155이며, 그 branch의 factor, mask, 보조 다항식 평가 및 RSA 재암호화 검증을 성공했다.

2 문제와 유출 구조

2048비트 RSA modulus $N = pq$, 공개 지수 $e = 65537$, textbook RSA 암호문 ct 가 주어진다. 1024비트 소수 p 에 대해서는 MASK와 $p \& \text{MASK}$ 도 공개된다. MASK가 1인 비트는 p 의 값이 알려졌고, 0인 비트만 미지수다.

LSB를 0번 비트로 두고 파싱하면 p 의 1024비트 중 672비트가 알려지고 352비트가 다음 일곱 구간에 나뉘어 있다.

구간	길이	역할
150-153	4	low edge 전수조사
265-348	84	첫 grouped 변수에 포함
362-419	58	첫 grouped 변수에 포함
600-668	69	둘째 grouped 변수에 포함
682-768	87	둘째 grouped 변수에 포함
784-829	46	둘째 grouped 변수에 포함
920-923	4	high edge 전수조사

352비트를 그대로 열거하면 $2^{352} \approx 9.17 \times 10^{105}$ 개다. 초당 10^{12} 개를 검사한다고 비현실적으로 낙관해도 약 2.9×10^{86} 년이 필요하다. 반면 양끝 두 구간은 합쳐서 8비트뿐이므로 이 부분만 256개로 열거하고, 큰 내부 구간은 격자 공격으로 넘긴다.

3 최종 공격

3.1 256개 edge 후보와 두 변수 완화

양끝 nibble를 다음처럼 하나의 후보 번호로 묶는다.

```
low  = p[150..153]
high = p[920..923]
cid  = (high << 4) | low
```

한 후보를 고정한 뒤 265...419와 600...829를 각각 연속 정수 x, y 로 둔다. 두 구간 안에는 이미 알려진 bit island도 있지만, 그 제약을 버려 변수 수를 줄인다.

$$p = C + 2^{265}x + 2^{600}y, \quad 0 \leq x < 2^{155}, \quad 0 \leq y < 2^{230}.$$

C 는 해당 edge nibble과 두 grouped 구간 밖의 알려진 비트를 합친 상수다. known-bit 제약을 완화했어도 실제 p 의 (x_0, y_0) 는 이 상자 안에 남는다. 이는 실제 근을 모델에서 제거하지 않았다는 뜻이지, 뒤의 휴리스틱 root extraction이 모든 branch의 근을 반드시 찾는다라는 뜻은 아니다.

3.2 이변수 Herrmann-May 격자

$A = 2^{265}, B = 2^{600}$ 으로 두고 $A^{-1} \bmod N$ 을 곱해 x 에 대해 monic인 선형식을 만든다.

$$f(x, y) = x + (BA^{-1} \bmod N)y + (CA^{-1} \bmod N) \pmod{N}.$$

실제 근에서 $C + Ax_0 + By_0 = p \mid N$ 이므로 $f(x_0, y_0) = 0 \bmod p$ 다. 복원된 p 는 약 $N^{0.500038}$ 인 1024비트 divisor이며 실제 근은

$$\text{bitlen}(x_0) = 155, \quad \text{bitlen}(y_0) = 228$$

이다. 선언한 $X = 2^{155}, Y = 2^{230}$ 안에 각각 0비트와 2비트의 여유로 들어온다.

$m = 17, t = 5$ 에서 다음 triangular shift를 사용한다 [4].

$$y^j f(x, y)^k N^{\max(t-k, 0)}, \quad k + j \leq m.$$

차원은

$$\frac{(m+1)(m+2)}{2} = \frac{18 \cdot 19}{2} = 171$$

이다. 실제 근에서 f^k 는 p^k 의 배수이고, $k < t$ 인 행에는 N^{t-k} 가 들어가므로 모든 shift 평가값은 최소 p^t 의 배수다. 감축 후 스케일을 되돌린 다항식 g 가

$$\|g(xX, yY)\|_1 < p^t$$

를 만족하면 $|g(x_0, y_0)| < p^t$ 이면서 $p^t \mid g(x_0, y_0)$ 이므로 정수 관계 $g(x_0, y_0) = 0$ 을 얻는다.

계수는 해당 N 거듭제곱을 기준으로 centered residue에 가깝게 만들고, monomial $x^i y^j$ 열을 $X^i Y^j$ 로 스케일한다. 이 171차원 basis를 목표 RHF 1.15로 FLATTER에서 감축했다 [17].

삼각 basis의 determinant는

$$\det(L) = X^{969} Y^{969} N^{250}$$

이다. 171차원에서 $\log_2(\det(L)^{1/171}) \approx 5175.58$ 이고 RHF 1.15를 단순 적용한 예상치는 약 5210.06비트인 반면 $\log_2(p^t) \approx 5119.99$ 다. 따라서 determinant 예상치만으로 성공을 보장하지 않는다. 실제 cid=155에서 선택한 12개 행의 scaled L1 bit length는 5107-5109였고 p^t 보다 약 11비트 이상 작았다. solver는 복원한 근을 이 12개 정수 다항식에 정확히 대입해 모두 0인지 추가 확인한다.

파라미터	값
m, t	17, 5
dimension	171
leading variable	x
centered coefficients	true
target RHF	1.15

3.3 Modular resultant, GCD와 CRT

감축된 행 중 L1 norm이 작은 12개를 보조 다항식으로 선택한다. 큰 정수 위에서 결과식을 한 번에 계산하는 대신 FLINT의 소수체 다항식 연산을 사용해 약 30비트 소수체에서 다음 과정을 반복한다 [18].

1. 보조 다항식 쌍의 resultant로 x 를 제거한다.
2. 여러 resultant의 GCD로 y 에 대한 일차식을 얻는다.
3. 얻은 y 를 대입한 보조 다항식들의 GCD로 x 를 얻는다.
4. (x, y) 가 선택한 모든 보조 다항식을 만족하는지 검사한다.
5. 서로 다른 소수체의 근을 CRT로 합친다.

오답 edge 후보는 대개 첫 소수 1,000,000,007에서 resultant GCD가 1이 되어 탈락한다. 정답 후보에서는 8개 소수의 공통근이 일치했고, CRT modulus가 약 240비트가 되어 Y 를 넘었을 때 정수 근과 p 가 유일하게 복원되었다. 구현도 종료 조건을 $\text{modulus} > \max(X, Y)$ 로 검사한다.

12개 행 선택, 일부 다항식 쌍만 쓰는 resultant, 일차 GCD만 처리하는 추출, 최대 10개 소수 제한은 heuristic이다. 후보가 나오면 $p \mid N$ 을 검사하므로 잘못된 factor를 채택하지 않지만, no_recovery가 나온 branch에 근이 없음을 증명하지는 않는다.

3.4 복원 결과와 RSA 복호화

성공 branch는 다음과 같다.

```
cid = 155 = 0x9b
low = 0xb
high = 0x9
```

두 소인수는 다음과 같다.

```
p =
ffa360d46885c534d186538179633fafc2c0548a2e24a2c1c878569f522939e3
8ff75ead1bcd442a834974a52e3ac66c1bc2ee00d63e2de4c436d2ca740a6246
99e1a1af94045c63261323cb723a7ba1b3c00bbb6a4e534c11469e73ddbb2e5
0b0bc2461fcbac0726360c2c0ac9450a9a892cbf1d98ceee48827591ccc593c9

q =
e557fa8670389cb60c84416a65742a74fd11ed33b1631f787e92b90887b5391d
acba00a386911bf8a8fbd57430b9b26e455329405ffe289e20616fe3b5562ea9
b533f8f8db94bb8dcd280a6af056108e176008d3655428ad0ac6396318ba0f6e
fe496eac3f8585675bfed67081e0c518be5685e4daf7060abe1c58b73cc5f1e9
```

그 뒤 표준 RSA primitive [16]에 따라

$$\varphi(N) = (p-1)(q-1), \quad d = e^{-1} \bmod \varphi(N), \quad m = ct^d \bmod N$$

을 계산해 첫 절의 평문을 얻었다.

4 정확성 경계

양성 결과의 soundness. 복원된 후보에 대해 $pq = N$, mask 일치, 근의 bound, 선택한 12개 보조 다항식의 정수 평가값 0, RSA 재암호화가 모두 독립적으로 확인된다. 따라서 보고한 factor와 평문이 맞다는 결론은 격자 휴리스틱의 성공률에 의존하지 않는다.

음성 결과의 completeness. FLATTER 감축, 12개 행 선택, 일부 resultant 쌍, 일차 GCD 추출은 완전 탐색이 아니다. 따라서 어떤 branch가 no_recovery를 반환해도 그 branch에 근이 없다는 뜻으로 사용하지 않는다. 전체 blind runner 역시 timeout/error가 하나라도 있거나 검증된 서로 다른 factor가 정확히 하나가 아니면 성공으로 판정하지 않도록 설계했다.

5 재현과 검증

5.1 결과 검증과 복호화

저장소 루트에서 다음 명령은 저장된 factor를 이용해 $pq = N$, mask 일치, RSA 재암호화를 검사하고 평문을 출력한다.

```
python3 solutions/solve_07_final.py
```

5.2 격자 공격 재실행

검증 환경은 Debian 12 계열, GCC 12.2.0, FLINT 2.9.0, GMP 6.2.1, OpenBLAS 0.3.21, fplll 5.4.4, MPFR 4.2.0이다. FLATTER revision은 다음으로 고정했다.

```
d2b8026f29b4a69e987b15d4b240f8a5053275d3
```

FLATTER를 해당 revision에서 Release build하고 설치 prefix를 \$FLATTER_PREFIX로 두면 다음처럼 빌드한다.

```
g++ -O3 -std=c++17 \
  -DFLATTER_GIT_COMMIT="'d2b8026f29b4a69e987b15d4b240f8a5053275d3'" \
  -I"$FLATTER_PREFIX/include" \
  solutions/solve_07_grouped_hm_flatter.cpp \
  -L"$FLATTER_PREFIX/lib" -Wl,-rpath,"$FLATTER_PREFIX/lib" \
  -lflatter -lflint -lgmpxx -lgmp -lopenblas -pthread \
  -o /tmp/solve_07_grouped_hm_flatter

/tmp/solve_07_grouped_hm_flatter --modular-selftest

OPENBLAS_NUM_THREADS=1 /tmp/solve_07_grouped_hm_flatter \
  --challenge --cid 155 --m 17 --t 5 --rhf 1.15 \
  --threads 4 --lead x --centered
```

마지막 명령은 이미 알려진 정답 branch cid=155를 다시 실행한다. 검증 환경의 두 실행에서 감축은 약 62-73초가 걸렸다. 정답 branch를 모르는 상태의 전체 탐색 진입점은 다음과 같다.

```
python3 solutions/run_07_grouped_hm_scan.py \
  /tmp/solve_07_grouped_hm_flatter \
  --output-dir /tmp/challenge07-full-scan \
  --jobs 2 --solver-threads 4 \
  --flatter-commit d2b8026f29b4a69e987b15d4b240f8a5053275d3
```

runner는 0부터 255까지를 모두 계획하며 정답 cid를 입력이나 실행 순서에 seed하지 않는다. 각 branch의 return code, wall time, stdout SHA-256, factor와 mask 검증을 JSON으로 남기고 binary/source SHA-256, compiler, ldd, 파라미터와 성공 후보 수를 summary에 모은다. returncode=2는 no_recovery일 뿐 무근 증명이 아니다. 이 보고서 작성 시 전체 256개 실행을 다시 완료하지 않았으므로, runner의 설계와 실제 재검증한 cid=155 결과를 구분한다.

6 실패한 전략과 판단 근거

아래에서 “실패”는 기법 자체가 틀렸다는 뜻이 아니라, 기록한 변수 분할과 파라미터에서 검증된 factor를 얻지 못했다는 뜻이다. UNSAT이나 INFEASIBLE, 정리의 bound와 안전 margin을 통과한 no-root, heuristic no-root, UNKNOWN/timeout을 구분했다.

6.1 1. 352비트 전수조사

아이디어. 숨은 비트를 모두 열거하고 각 후보에 대해 $p \mid N$ 을 검사하는 가장 직접적인 기준선이다 [1]. **실제 관측:** 후보 수는 2^{352} 이며, 가장 작은 내부 46비트 구간만 열거해도 약 7.0×10^{13} 개다. 그 구간 하나를 정해도 나머지 구간은 결정되지 않는다. 계산량이 현실적인 범위를 압도하므로 전체 공격으로 채택하지 않았다. 다만 양끝 8비트만은 256개이므로 최종 공격의 edge enumeration으로 제한해 사용했다. **배제 증거:** 열거하지 않은 공간을 배제하는 증거는 전혀 없으며, 단순한 복잡도 평가다.

6.2 2. 전체 2048비트 곱셈 SMT

아이디어. 1024비트 p, q 와 $pq = N$ 을 bit-vector로 한 번에 표현하고 누출 bit 및 유도한 q prefix를 함께 전파한다. Z3의 bit-vector SMT를 사용했다 [7]. **실제 관측:** p 672비트와 q 250비트를 고정한 full-size 실행이 약 63초 뒤 UNKNOWN으로 끝났다. 흠어진 미지 구간의 부분곱과 carry가 먼 known-bit island까지 전달되어야 해 제한 시간 안에 factor나 판정을 만들지 못했다. **배제 증거:** UNKNOWN은 UNSAT이 아니므로 어떤 branch도 배제하지 못한다.

6.3 3. 하위 곱셈 exact-tail CP-SAT

아이디어. 전체 곱 대신 $pq = N \bmod 2^T$ 의 limb 부분곱과 carry만 정확히 모델링해 상태를 줄인다. 사용한 solver 계열은 CP-SAT-LP다 [8]. **실제 관측:** $T = 928$ 모델의 16회 얇은 탐색은 195.58초 동안 247,599 branches와 14,095 conflicts를 만들었지만 모두 UNKNOWN이었다. 48회 30초 탐색도 총 1,504.11초, 852,177 branches, 77,505 conflicts 뒤 전부 UNKNOWN이었다. 꼬리 해가 완전한 factor를 보장하지도 않고, 실행 시간을 늘려도 유용한 판정력이 생기지 않아 direct-hit 진단으로만 남겼다. **배제 증거:** CP-SAT의 INFEASIBLE만 안전한 배제인데 실제로는 UNKNOWN뿐이었다.

6.4 4. Hensel-prefix 분기

아이디어. $pq = N \bmod 2^k$ 를 만족하는 하위 prefix에 bit를 하나씩 붙이고 새 known bit와 충돌하는 가지만 버린다. RSA key-bit branch-and-prune 연구와 관련된다 [9]. **실제 관측:** top 16의 $k = 370, 390, 410$ 검사는 52.68초가 걸렸고, 370비트에서 SAT 15개/UNKNOWN 1개, 390과 410비트에서는 모두 UNKNOWN이었다. top 4의 $k = 430, 500, 600$ 도 70.64초 뒤 모두 UNKNOWN이었다. 긴 미지 구간에서 다음 known island에 닿기 전에 상태가 커져 유용한 모순을 찾지 못했다. **배제 증거:** UNSAT이라면 해당 modulus prefix를 배제할 수 있지만 관측되지 않았다. SAT은 완전 factor의 존재를 뜻하지 않고 UNKNOWN도 결론을 주지 않는다.

6.5 5. Exact 다변수 Herrmann-May

아이디어. 일곱 미지 구간을 각각 변수로 보존해 unknown divisor modulo의 선형 작은 근을 직접 찾는다 [4]. **실제 관측:** 7변수 $m = 4, t = 1$ 은 후보 하나에 약 340초 뒤 실패했고 $m = 4, t = 2$ 는 1,200초 제한에도 끝나지 않았다. edge를 고정한 5변수 $m = 2, t = 1$ 은 256개를 10.5초에 훑었지만 factor가 없었고 $m = 3, t = 1$ 은 후보 하나도 300초 timeout이었다. 낮은 차수는 너무 약하고 한 단계만 올려도 monomial과 격자 비용이 폭증해 256개 edge 후보에 적용할 수 없었다. **배제 증거:** 낮은 파라미터의 no-factor와 timeout은 격자 휴리스틱의 관측이며 branch의 UNSAT 증명이 아니다.

6.6 6. 얇은 grouped Herrmann-May

아이디어. known-bit island 일부를 완화해 변수 수를 둘로 낮추되 각 변수 bound를 키운다. Herrmann-May의 다중-block 관점에 기반한다 [4]. **실제 관측:** [265, 769) 504비트와 [784, 830) 46비트 분할, 그리고 84비트/468비트 분할에서 $m = 1 \dots 8, t = 1, 2$ 의 4,096개 작업과 $m = 10$ 의 512개를 더 실행했지만 factor가 없었다. $m = 14$ 는 오답 표본 하나에도 73-106초가 들었다. 변수 크기가 지나치게 불균형한 얇은 basis는 관계를 드러내지 못했다. 그러나 grouping 자체는 버리지 않고 155/230비트의 더 균형 잡힌 분할과 $m = 17, t = 5$ 의 강한 basis로 바꾸어 최종 성공 경로가 되었다. **배제 증거:** 얇은 sweep의 no-factor는 branch 배제가 아니다.

6.7 7. cuso mixed-shape

아이디어. grouped, exact, mixed 변수 shape를 한 실행기에 넣고 자동 shift 구성과 graph heuristic으로 격자 크기를 줄이려 했다. cuso 연구와 artifact를 참고했다 [12, 13]. **실제 관측:** $S0=[155,230]$, $S1=[84,58,69,87,46]$, $S2=[84,58,230]$, $S3=[155,69,87,46]$, $S4=\text{low600}$ 의 설계를 만들었지만, planted/toy root-extraction soundness gate를 통과한 완료 sweep까지 진행하지 못했다. 외부 실행 환경과 추출 경로가 검증되기 전에 나온 상태를 수학적 결과로 해석할 수 없어 보류했다. **배제 증거:** 검증된 no-root가 없으므로 hard clause는 물론 soft no-root로도 사용하지 않았다.

6.8 8. 단변수 two-sided p window

아이디어. 알려진 하위와 상위 비트 사이를 하나의 연속 변수로 두어 단변수 Coppersmith/Howgrave-Graham 공격을 적용한다 [2, 3]. **실제 관측:** 468비트 window는 첫 후보가 120초를 넘겼고, 504비트 gap도 상위 후보가 90초 timeout이었다. 410비트로 줄인 변형도 후보당 약 60초가 필요했다. 단변수의 단순함보다 410-504비트의 root bound가 더 큰 병목이었고, 넓은 candidate oracle로 쓰기에는 너무 비쌌다. **배제 증거:** timeout은 window에 근이 없다는 증거가 아니다.

6.9 9. Residual partial- p 격자

아이디어. 선행 ranker가 외부 비트를 맞혔다고 가정하고 남은 2-3개 구간에만 작은 Herrmann-May 격자를 적용한다 [4]. **실제 관측:** 69/87/46비트 3변수에 대해 32개 후보와 $m = 2 \dots 4, t = 1 \dots 4$ 를 시험했지만 factor가 없었고 $m =$

4, $t = 1$ 은 모두 timeout이었다. 후속 128개씩의 sweep과 58/69/87비트 분할의 상위 후보도 검증된 factor를 만들지 못했다. 검증되지 않은 ranker가 정답 근처를 고른다는 가정에 민감해 넓은 탐색보다 선택 편향만 키웠다. **배제 증거:** 선정 후보의 no-factor 또는 timeout일 뿐, 미시적 공간이나 해당 branch를 배제하지 않는다.

6.10 10. Folded p/q Coron 격자

아이디어. p 와 q 의 gap을 각각 변수로 두고 정수 이변수식 $p(x)q(y) - N = 0$ 에서 두 번째 관계를 찾는다 [5]. **실제 관측:** 양쪽 bound가 각각 456비트인 모델에서 1,024개 후보와 $k = 1 \dots 4$ 를 444.0초에 검사했지만 factor가 없었다. $k = 5$ 부터는 후보 하나에 수분이 필요했다. 낮은 k 는 너무 약하고 broad sweep에서 k 를 높이는 비용은 비현실적이라 finalist 교차 검사로만 남겼다. **배제 증거:** 작은 k 의 no-factor는 선택한 basis가 근을 드러내지 못했다는 뜻이며 무근 증명이 아니다.

6.11 11. q-gap 단변수 oracle

아이디어. 일부 p 비트로 유도한 q 의 연속 하위·상위 prefix 사이를 하나의 변수로 두고 단변수 small-root oracle을 적용한다 [2, 3, 15]. **실제 관측:** 여러 candidate에서 빠른 no-root가 나와 직접 탐색과 후속 SAT clause oracle로 사용했지만, 사후에 복원된 q 를 대조하면 $q^2 < N$ 인데 Sage 호출은 $\beta = 0.5$ 를 사용했다. API의 divisor 조건 $b \geq N^\beta$ 가 실제 작은 소인수 q 에 충분하지 않아 no-root를 안전한 결론으로 해석할 수 없었다. **배제 증거:** 기록된 q-gap no-root는 모두 heuristic이며 hard branch 배제에 쓰지 않았다.

6.12 12. q-gap SAT learned clause

아이디어. SAT가 partial assignment를 만들고 q-gap oracle이 모든 completion을 부정하면 외부 loop가 no-good clause를 추가한다. SAT와 격자를 결합한 인수분해 연구와 관련된다 [10]. **실제 관측:** completion 전수검사와 clause 최소화를 구현해 ledger를 늘렸지만, 앞 절의 q-gap oracle 자체가 $\beta = 0.5$ 조건을 만족하지 않았다. ledger가 커질수록 다음 모델 선택 비용도 증가했다. oracle의 음성 판정이 sound하지 않으므로 completion을 모두 검사해도 clause가 hard 증거가 되지 못했고, 정답 방향을 주는 목적 함수도 없었다. **배제 증거:** 이 실행의 q-gap clause는 heuristic only다.

6.13 13. low600 p -Coppersmith clause

아이디어. 하위 600비트를 고정해 $p = p_0 + 2^{600}x$, $0 \leq x < 2^{424}$ 인 단변수 문제를 만들고, 검증된 no-root cube를 SAT clause로 바꾼다 [2, 3, 10, 15]. **실제 관측:** Sage backend에서 effective bound 470.04비트, 실제 상한 424비트, margin 46.04비트였다. 16/16 completion으로 142-literal clause를, 256/256 completion과 273회 oracle 호출 및 약 451초로 138-literal clause를 얻었다. 후속 cumulative 실행은 첫 cube row도 쓰기 전에 약 5.5시간 정체되어 중단됐다. 큰 소인수 p 는 $p^2 > N$ 이라 $\beta = 0.5$ 조건을 만족해 얻은 두 clause 자체는 sound했지만, 한 건의 비용에 비해 닫히는 146비트 cube 공간이 너무 작아 전체 탐색을 진전시키지 못했다. **배제 증거:** backend, bound, margin 및 모든 completion을 통과한 두 clause의 범위 안에서만 hard 배제다. 불완전한 후속 로그는 증거로 쓰지 않았다.

6.14 14. SAT ranker와 다양성 샘플링

아이디어. deep-prefix/부분급 점수, guarded cycle, projection novelty와 compacted ledger로 다음 SAT 모델의 순서와 다양성을 개선한다. 다양한 SAT 해 생성 연구는 동기를 제공한다 [11]. **실제 관측:** 9,579개 clause와 1,319,382 literals를 읽은 ranker는 후보당 250ms에서 UNKNOWN만 냈고, 4,096-pair Z3 ranker도 약 6분 동안 CPU-bound여서 중단했다. projection novelty는 중복 모양을 줄였지만 factor 확률과의 상관은 확인되지 않았다. “아직 보지 않은 후보”는 만들었지만 “정답에 가까운 후보”라는 신호를 제공하지 못해 최종 공격의 후보 선택기로 채택하지 않았다. **배제 증거:** 순위와 다양성은 확률적 heuristic이며 어떤 branch도 배제하지 않는다.

6.15 15. Jochemsz–May low-lift

아이디어. monomial별 bound를 반영한 다변수 shift를 감축해 12항 2차 다항식의 공통 작은 근을 찾는다 [6, 14]. **실제 관측:** $Wbits = 1591$ 에서 $m = 1, 2$ 및 확장 설정 ext_u2, ext_b가 crypto-attacks의 기본 resultant root extraction에서 IndexError로 끝났다. 확장 설정은 각각 57.5초와 85.8초를 썼다. 이는 격자가 근을 갖지 않는다는 결과가 아니라 구현·추출 경로가 후보를 내지 못한 오류다. planted 회귀 테스트로 추출을 먼저 검증하지 못했으므로 진단 경로로만 남겼다. **배제 증거:** 구현 오류는 no-root나 branch 배제 증거가 아니다.

7 최종 검증 항목

```
p * q == N
(p & MASK) == P_AND_MASK
pow(m, 65537, N) == ct
m.hex() ==
464c41477b64317274795f6231745f6c33346b5f633070703372736d3174685f
6d333374735f73747234743367797d
```

8 참고문헌

References

- [1] R. Rivest, A. Shamir, and L. Adleman, *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*, Communications of the ACM, 1978. <https://people.csail.mit.edu/rivest/pubs/RSA78.pdf>
- [2] D. Coppersmith, *Small solutions to polynomial equations, and low exponent RSA vulnerabilities*, Journal of Cryptology, 1997. <https://research.ibm.com/publications/small-solutions-to-polynomial-equations-and-low-exponent-rsa-vulnerabilities>
- [3] N. Howgrave-Graham, *Finding Small Roots of Univariate Modular Equations Revisited*, Cryptography and Coding, LNCS 1355, 1997. <https://doi.org/10.1007/BFb0024458>
- [4] M. Herrmann and A. May, *Solving Linear Equations Modulo Divisors: On Factoring Given Any Bits*, ASIACRYPT 2008. <https://www.iacr.org/archive/asiacrypt2008/53500412/53500412.pdf>
- [5] J.-S. Coron, *Finding Small Roots of Bivariate Integer Polynomial Equations Revisited*, EUROCRYPT 2004. <https://iacr.org/archive/eurocrypt2004/30270487/bivariate.pdf>
- [6] E. Jochemsz and A. May, *A Strategy for Finding Roots of Multivariate Polynomials with New Applications in Attacking RSA Variants*, ASIACRYPT 2006. <https://www.iacr.org/archive/asiacrypt2006/42840270/42840270.pdf>
- [7] L. de Moura and N. Bjørner, *Z3: An Efficient SMT Solver*, TACAS 2008. https://doi.org/10.1007/978-3-540-78800-3_24
- [8] L. Perron, F. Didier, and S. Gay, *The CP-SAT-LP Solver*, CP 2023. <https://doi.org/10.4230/LIPIcs.CP.2023.3>
- [9] N. Heninger and H. Shacham, *Reconstructing RSA Private Keys from Random Key Bits*, CRYPTO 2009. <https://hovav.net/ucsd/papers/hs09.html>
- [10] Y. Ajani and C. Bright, *SAT and Lattice Reduction for Integer Factorization*, ISSAC 2024. <https://doi.org/10.1145/3666000.3669712>
- [11] A. Nadel, *Generating Diverse Solutions in SAT*, SAT 2011. https://doi.org/10.1007/978-3-642-21581-0_23
- [12] K. Ryan, *Solving Multivariate Coppersmith Problems with Known Moduli*, EUROCRYPT 2025. <https://eprint.iacr.org/2024/1577.pdf>
- [13] K. Ryan, *EUROCRYPT 2025 cuso Artifact*. <https://artifacts.iacr.org/eurocrypt/2025/a13/readme.html>
- [14] jvdsn, *crypto-attacks: Python implementations of attacks on cryptographic constructions*. <https://github.com/jvdsn/crypto-attacks>
- [15] SageMath, *small_roots API documentation*. https://doc.sagemath.org/html/en/reference/polynomial_rings/sage/rings/polynomial/polynomial_modn_dense_ntl.html
- [16] K. Moriarty et al., *PKCS #1: RSA Cryptography Specifications Version 2.2*, RFC 8017. <https://www.rfc-editor.org/rfc/rfc8017.html>
- [17] K. Ryan, *FLATTER: Fast Lattice Reduction*. <https://github.com/keeganryan/flatter>
- [18] FLINT project, *nmod_poly documentation*. https://flintlib.org/doc/nmod_poly.html